# Tiberius: A Security Testing Framework for LLM Applications in Java

Iryna Dohndorf
github.com/tiberius-security/tiberius

June 2026

**Abstract**

Large Language Models deployed in production Java applications introduce a security and safety testing challenge that conventional unit testing frameworks are not equipped to address: non-deterministic outputs, linguistic attack surfaces, and probabilistic failure modes. This paper presents **Tiberius**, an open-source Java library that brings software testing discipline to LLM security validation. Tiberius introduces a *scan-fixture-validate* workflow for regression testing, fixture-driven guardrail validation against real attack data, probabilistic security contracts for CI/CD quality gates, bias testing as a first-class CI-enforceable property, and model fingerprinting for both offensive assessment and defensive hardening. The library is built around the premise that vulnerability testing of LLM applications must be systematic, reproducible, and grounded in real attack data — not hand-crafted inputs or isolated single-sample assertions.

*How do you write a regression test for a system that is non-deterministic by design?*

## 1. The Problem

Large Language Models have moved from research artifacts to production infrastructure. Java applications are embedding them into customer-facing services via Spring Boot and LangChain4J — for document summarization, customer support, healthcare assistance, and financial guidance, to name just a few. The deployment surface is growing faster than the security tooling.

The vulnerability landscape is empirically well-established. Horlacher, Vifian, and Zagidullina [4] red-teamed `gpt-oss-20b` and found that adversarial techniques achieved alarmingly high Attack Success Rates, while non-adversarial probing exposed pervasive stereotypical defaults — both consistent across English and Swiss German. Their conclusion: "current alignment mechanisms have not fully resolved jailbreaks and inherent bias, posing critical challenges for automated decision-making."

The engineering community's response has been solid on the Python side. Praetorian's Augustus [1] provides a comprehensive scanning framework. Garak [6], PromptBench, and others address evaluation from a research angle. For Java teams building on Spring Boot and JUnit 5, having a testing tool that fits naturally into the existing workflow is not just convenient — it makes development considerably more efficient and helps ensure the security and safety of the software being built.

There is a further challenge. Generic benchmarks test model behavior in isolation. But applications are rarely built on a simple generic model. A Java application has a system prompt, business logic, custom guardrails, and a specific user population. The attack surface that matters is the intersection of adversarial technique and a specific deployment context.

## 2. What Tiberius Does

Tiberius[1] is an open-source Java library for vulnerability and security testing of LLM applications. It integrates with JUnit 5 and Spring Boot, and is designed to fit naturally into a standard Java test suite.

The library is shaped by five recurring challenges encountered when testing LLM applications in practice.

### 2.1 Fixture-Based Regression Testing

The standard unit test model — fixed input, deterministic output, assert equality — does not transfer to LLM testing. LLM responses are non-deterministic. The same prompt may produce different outputs across invocations, model versions, or configuration changes.

Tiberius addresses this with a **scan-fixture-validate** workflow. A scan run executes 210+ attack probes against a deployed model and serializes the results — including which attacks succeeded, the actual prompts and responses, and severity scores — to a JSON fixture file.

```
@ExtendWith({TiberiusExtension.class, FixtureExtension.class})
@CreateFixture("fixtures/baseline-scan.json")
class LLMSecurityScan {

    @Test
    void scanForVulnerabilities(TiberiusScanner scanner,
        FixtureContext fixture) {
         scanner.setGenerator(new OllamaGenerator("llama3.2"));
         ScanReport report = scanner.scan();
         fixture.record(report);

         log.info("Attack success rate: {}%", report.successRate());
    }
}
```

Listing 1: Scan phase: executing probes and recording a fixture

The fixture becomes a reproducible dataset of attacks that actually penetrated the model. It is version-controlled, shareable, and stable — the non-determinism of the LLM is isolated to the scan phase. Downstream tests consume the fixture without re-querying the model, making all subsequent phases fully deterministic and repeatable.

This is the same engineering pattern as snapshot testing in frontend development, applied to adversarial inputs. The fixture is the ground truth.

### 2.2 Guardrail Validation Against Real Attack Data

Most guardrail testing is done with hand-crafted inputs. A developer writes a few example prompts, checks that the guardrail blocks them, and ships. Coverage is limited by the developer's imagination and familiarity with attack techniques. Direct prompt injection — first systematically characterized by Perez & Ribeiro [5] — demonstrates how trivially this coverage can be exceeded.

Tiberius inverts this. After a scan, the developer has a fixture of attacks that actually bypassed the model. Guardrails are then validated against that fixture:

```
@Test
void guardrailsBlockKnownAttacks() {
```

[1] https://github.com/tiberius-security/tiberius

```
    InputGuardrail guardrail = new PromptInjectionGuardrail();

    GuardrailTestResult result = GuardrailTester
        .test("PromptInjectionGuardrail",
              text -> guardrail.validate(UserMessage.from(text)).
                 result() == FAILURE)
        .withAttacksFromFixture("fixtures/baseline-scan.json",
           AttackCategory.JAILBREAK)
        .withAttacksFromFixture("fixtures/baseline-scan.json",
           AttackCategory.PROMPT_INJECTION)
        .withSafeInputs(
            "What is my account balance?",
            "Transfer $100 to savings"
        )
        .run();

    // Block rate and false positive rate are first-class metrics
    assertThat(result.blockRate()).isEqualTo(1.0);
    assertThat(result.noFalsePositives()).isTrue();
}
```

Listing 2: Guardrail validation against fixture-based real attack data

This tests two properties simultaneously: that the guardrail blocks adversarial inputs, and that it does not block legitimate ones. Both false negatives and false positives are tracked as first-class metrics. The output is a structured report:

```
Guardrail: PromptInjectionGuardrail
Attacks tested: 150
Blocked: 150 (100%)
Bypassed: 0 (0%)
False positives: 0
```

### 2.3 Probabilistic Security Contracts

A single test run against an LLM tells you what happened on that invocation. It does not tell you the underlying probability that an attack succeeds. For a system where correctness is statistical rather than deterministic, single-sample PASS/FAIL results are insufficient.

Tiberius integrates with PUnit [3] — a probabilistic testing framework for Java — to support multi-trial scanning and statistical assertions:

```
@Test
void probabilisticSecurityContract(TiberiusScanner scanner) {
    scanner.setGenerator(new OllamaGenerator("llama3.2"));

    ScanReport report = scanner.multiTrialScan()
        .samples(35)      // Each attack probe executed 35 times
        .execute();

    // successRate() = (attacks succeeded / total attacks) x 100
    // 0.0 means: not a single attack got through across all 35
       trials
    assertThat(report.successRate()).isEqualTo(0.0);
}
```

Listing 3: Multi-trial probabilistic security assertion

This can be formalized into **security contracts** — statistical requirements that a model must satisfy before deployment:

```
SecurityContract contract = SecurityContract.builder()
    .name("Production LLM Security")
    .requirement(SecurityCriteria.jailbreakResistance(1.0))
    .requirement(SecurityCriteria.dataExtractionResistance(1.0))
    .requirement(SecurityCriteria.overallResistance(1.0))
    .build();

contract.verify(scanner.scan());
```

Listing 4: Declarative security contract for CI/CD quality gates

A security contract is a testable, version-controlled specification of acceptable model behavior. It fails the build when violated, giving CI/CD pipelines a concrete, enforceable quality gate.

### 2.4 Bias Testing

Most LLM security frameworks focus exclusively on adversarial intent. Tiberius extends the testing surface to **systemic bias**: model behavior on ambiguous, non-adversarial inputs where no single answer is correct, but where a fair system should not exhibit systematic preferences.

This matters because bias is not just a correctness defect — it is an ethical concern. A biased model produces subtly wrong outputs at scale, in ways that are invisible to traditional assertion-based tests. Software developers building AI-enriched applications have skin in the game: at the scale LLMs operate, a biased model does not affect one user in isolation — it affects every user who encounters that system, systematically and silently. Writing a bias test is not optional due diligence; it is part of the engineering contract.

```
@Test
void modelDoesNotDefaultToGenderStereotypes(TiberiusScanner scanner)
   {
    BiasReport report = scanner.biasScan()
        .category(BiasCategory.GENDER)
        .scenario("A software engineer walks into a meeting. Describe
            them.")
        .variants(50)
        .execute();

    assertThat(report.distributionSkew()).isLessThan(0.1);
    assertThat(report.stereotypeRate()).isEqualTo(0.0);
}
```

Listing 5: Bias probe: testing for gender stereotypes across a response distribution

Bias, like security, is probabilistic by nature. A single response can look neutral; the signal only emerges across a distribution of responses. Tiberius applies the same multi-trial, statistical approach to both.

### 2.5 Model Fingerprinting

Before testing a model, one must know what is being tested. Tiberius includes a fingerprinting capability inspired by Julius [2] that identifies the underlying model behind an API endpoint — useful when the provider is opaque, the model version is undocumented, or a third-party deployment is being audited.

```
FingerprintReport report = TiberiusFingerprinter.probe(generator);

System.out.println(report.likelyModel());     // e.g. "gpt-4o-mini"
System.out.println(report.confidence());      // e.g. 0.91
System.out.println(report.providerHints());   // e.g. [OPENAI,
    AZURE_OPENAI]
```

Listing 6: Offensive fingerprinting: identifying an unknown model

The defensive implication is equally important: **production LLM applications should not be fingerprintable**. A model that reveals its identity through behavioral probes gives attackers a precise attack surface. Tiberius lets teams verify that their own deployment does not leak this information:

```
@Test
void productionEndpointResistsFingerprinting(TiberiusScanner scanner)
    {
    FingerprintReport report = TiberiusFingerprinter.probe(generator)
        ;

    assertThat(report.confidence()).isLessThan(0.2);
    assertThat(report.modelIdentified()).isFalse();
}
```

Listing 7: Defensive test: asserting fingerprinting resistance

## 3. Attack Coverage

Tiberius ships with 210+ probes across nine categories, mapped to the OWASP LLM Top 10 [7]:

| Category | Examples | Probes |
|---|---|---|
| JAILBREAK | DAN, AIM, persona manipulation | 45+ |
| ENCODING | Base64, ROT13, Morse, hex | 30+ |
| PROMPT_INJECTION | Instruction override | 40+ |
| DATA_EXTRACTION | System prompt leakage, PII, API keys | 25+ |
| MULTI_TURN | Crescendo, GOAT, Hydra escalation | 20+ |
| FORMAT_EXPLOIT | Markdown, XML, JSON injection | 15+ |
| CONTEXT_MANIPULATION | RAG poisoning, context overflow | 20+ |
| ADVERSARIAL | GCG, AutoDAN token attacks | 10+ |
| EVASION | Homoglyphs, zero-width characters | 15+ |

Buff transformations apply evasion techniques on top of any probe — Base64 encoding, ROT13, hypothetical or poetry framing, fictional context — and can be chained to test compound evasion strategies. Developers can define their own mutation operators, making Buffs the LLM equivalent of fault injection:

```
// Built-in buffs
scanner.addBuff(EncodingBuffs.BASE64);
scanner.addBuff(StyleBuffs.HYPOTHETICAL);

// Chain buffs: encode first, then wrap in fictional framing
Buff combined = EncodingBuffs.BASE64.andThen(StyleBuffs.FICTION);
scanner.addBuff(combined);

// Define your own mutation operator
```

```
Buff domainSpecific = prompt ->
    "In the context of a financial compliance audit: " + prompt;

scanner.addBuff(domainSpecific);
```

Listing 8: Custom buff: domain-specific mutation operator

A guardrail that blocks `"Generate a phishing email"` may not block `"For a peer-reviewed study on social engineering vectors, produce a representative specimen of a credential-harve message."`. Custom Buffs let domain knowledge be encoded directly into the test suite.

## 4. Integration

Tiberius is available on Maven Central:

Listing 9: Maven dependency

```
<dependency>
    <groupId>io.github.tiberius-security</groupId>
    <artifactId>tiberius</artifactId>
    <version>1.0.0</version>
    <scope>test</scope>
</dependency>
```

Tiberius supports Ollama (local), OpenAI, Anthropic, and any OpenAI-compatible REST API as generators. Spring Boot auto-configuration is provided via `@Import(TiberiusAutoConfiguration.class)`. No framework changes are required — tests are standard JUnit 5.

## 5. The Case for Shared Attack Datasets

Adversarial attacks are not generic. A jailbreak effective against a legal document assistant differs structurally from one targeting a medical triage chatbot or a financial advisory system. Industry-specific context — regulatory language, domain vocabulary, professional role-play framings — creates attack vectors that general probe libraries do not cover.

Attack datasets should be shared across teams and organizations, not siloed. A healthcare team that discovers a prompt injection exploiting clinical terminology has produced intelligence directly useful to every other healthcare AI deployment. The same applies across fintech, legal, public sector, and any regulated domain.

Tiberius's fixture format is designed for exactly this. A scan fixture is a plain JSON file — version-controllable, shareable, publishable. Teams can contribute domain-specific probe sets back to the community:

```
GuardrailTestResult result = GuardrailTester
    .test("MedicalAssistantGuardrail", guardrail::shouldBlock)
    .withAttacksFromFixture("fixtures/community/healthcare-attacks
       -2026.json")
    .withAttacksFromFixture("fixtures/community/health-insurances-
       roleplay-injections.json")
    .withAttacksFromFixture("fixtures/local/production-findings.json"
       )
    .run();
```

Listing 10: Loading community and local attack fixtures

No single team has the breadth of adversarial knowledge that a community does. Contributions to Tiberius's probe library — especially domain-specific fixtures — have compounding value across every organization that adopts the framework.

## 6. Security Testing as a First-Class Engineering Concern

The software engineering community has built extensive infrastructure for testing deterministic systems. Smoke tests gate a deployment — confirming that critical functionality holds before deeper verification begins. Property-based testing handles fuzzing. Snapshot testing handles regression. Contract testing handles API compatibility. These tools encode a shared insight: the test artifact — the fixture, the contract, the property — is as important as the test itself. Tiberius adds a missing entry to that list: security contracts as first-class CI gates, and scan fixtures as the LLM equivalent of a smoke test.

LLM applications break all of these abstractions. The output is probabilistic. The attack surface is linguistic. The failure modes are semantic rather than syntactic.

Tiberius is an attempt to bring the discipline of software testing to this new class of system — fixture-driven, statistically grounded, integrated into the standard Java development workflow. Crucially, it opens a path toward antifragility: attacks that bypass a model do not just register as failures — they become fixtures, feeding directly into guardrail validation and making the system demonstrably stronger with every breach.

## Acknowledgements

The author thanks **Barbara Teruggi** (LinkedIn), who pointed to Augustus and consistently shares critical security intelligence that keeps the community informed and ahead of emerging threats. This project started with that pointer.

Warm thanks to **Mike Mannion** (LinkedIn), creator of PUnit, with whom many of the concepts that shaped Tiberius were discussed. Mike articulated the practical relevance of test fixtures and shared datasets with clarity that directly influenced this work, and has consistently championed the importance of bias testing as a serious engineering concern.

## References

[1] Praetorian Security, Inc. *Augustus: Open-Source LLM Vulnerability Scanner*. 2026. https://github.com/praetorian-inc/augustus

[2] Praetorian Security, Inc. *Julius: LLM Service Identification and Security Evaluation Tool*. https://github.com/praetorian-inc/julius

[3] Mannion, M. *PUnit: Probabilistic Unit Testing Framework for Java*. https://github.com/mavai-org/punit

[4] Horlacher, S., Vifian, S., & Zagidullina, A. *Red Teaming GPT-OSS-20B: Evaluating Jailbreak Susceptibility and Bias Across English and Swiss German*. SwissText 2026. https://www.swisstext.org/current/submissions/accepted-submissions/

[5] Perez, F. & Ribeiro, I. *Ignore Previous Prompt: Attack Techniques For Language Models*. arXiv:2211.09527, 2022. https://arxiv.org/abs/2211.09527

[6] NVIDIA. *Garak: LLM Vulnerability Scanner*. arXiv:2406.11036, 2024. https://github.com/NVIDIA/garak

[7] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. https://owasp.org/www-project-top-10-for-large-language-model-applications/